

Build Smarter with Agentic Engineering

Participant Assessment • 10 multiple-choice questions • Answer key included for coordinators

Duration	15 minutes
Questions	10 multiple choice, exactly one correct answer each
Marking	1 mark per question. No negative marking.
Suggested pass	6 out of 10
Coverage	Agentic engineering practice, reviewing generated code, architecture trade-offs, containers, CI/CD and GitOps

Note for coordinators: these questions test understanding of *why* a choice was made, not recall of commands. Several distractors are plausible and commonly believed - that is deliberate, and a participant who reasons well about a wrong answer is worth noting.

Section A • Questions

Q1. Pulse refreshes every live view with a plain HTTP request every 2 seconds instead of using WebSockets. What is the primary engineering reason?

- A) It is faster than a WebSocket connection
- B) It converts a stateful problem into a stateless one, so a dropped request needs no recovery logic
- C) WebSockets cannot deliver JSON payloads
- D) It reduces the total number of network requests

Q2. The rule "one vote per device per poll" is enforced BOTH by an if-check in the request handler AND by a UNIQUE constraint in the database. Why is the if-check alone insufficient?

- A) The if-check is slower than a database constraint
- B) Database constraints produce friendlier error messages
- C) Two simultaneous requests can both pass the check before either writes, so only the constraint actually prevents a double vote
- D) The if-check does not work when the database is SQLite

Q3. What is the main purpose of an AGENTS.md file in a repository?

- A) To document the public API for end users
- B) To store environment variables the agent needs at runtime
- C) To replace the README for new contributors
- D) To state the stack, test commands and hard rules in a durable place every agent re-reads

Q4. When reviewing a feature written by a coding agent, the test suite is fully green. Why is that NOT sufficient evidence that the requirements are met?

- A) Automated tests always contain bugs
- B) The agent wrote the tests too, so a rule it forgot has no test and nothing reports the absence
- C) Green tests mean the code was never actually executed
- D) Test coverage percentage is a more reliable signal

Q5. In the GitOps model used in this session, what actually triggers a deployment to the cluster?

- A) A developer running kubectl apply against the cluster
- B) CI connecting to the cluster and pushing the new image directly

- C) An administrator clicking Deploy in a dashboard
- D) CI committing the new image tag to the repository, which ArgoCD observes and reconciles

Q6. A path-filtered CI pipeline reports a green check on a commit that changed only documentation. What does that green check actually mean?

- A) Every test in the repository passed
- B) The test results were reused from a cached previous run
- C) No relevant paths changed, so the test jobs were skipped and nothing ran
- D) The pipeline is misconfigured and should be fixed

Q7. What most distinguishes a usable specification from a prompt when handing work to a coding agent?

- A) It is longer and written in more formal language
- B) It names the programming language and framework explicitly
- C) It is written in Markdown rather than plain text
- D) It has numbered, individually checkable acceptance criteria, plus where code lands, what to reuse and what is out of scope

Q8. Why should a Kubernetes manifest reference a container image by commit SHA rather than the :latest tag?

- A) SHA-tagged images are downloaded faster by the kubelet
- B) The :latest tag is immutable and therefore cannot be rolled back
- C) A SHA tag is immutable, so you can say exactly what is running and roll back with confidence
- D) Kubernetes does not support the :latest tag

Q9. The Pulse web app only ever calls relative paths such as /api/polls, with no configurable API base URL. What does this design eliminate?

- A) The need for HTTPS in production
- B) A per-environment build, and the class of bugs where the wrong backend URL is shipped
- C) The need for an ingress controller or reverse proxy
- D) All cross-origin requests from mobile clients

Q10. Pulse identifies voters by a random device_id in localStorage, which any user can trivially reset. How should this design decision be characterised?

- A) A security vulnerability that must be fixed before release
- B) An oversight caused by time pressure during the build
- C) A deliberate trust boundary left undefended because the defence costs more than the threat
- D) An industry-standard authentication mechanism for web applications

Section B • Answer Key and Explanations

For coordinators only. Each explanation carries the reasoning, so partial credit can be judged if a participant argues a defensible case.

Q	1	2	3	4	5	6	7	8	9	10
Answer	B	C	D	B	D	C	D	C	B	C

Q1. Pulse refreshes every live view with a plain HTTP request every 2 seconds instead of using WebSockets. What is the primary engineering reason?

Correct answer: B) It converts a stateful problem into a stateless one, so a dropped request needs no recovery logic

Why: A WebSocket is long-lived and stateful: a drop needs reconnect logic, backoff, heartbeats and state resync. A dropped poll simply happens again two seconds later, because each request carries the whole answer. On hostile venue wifi the stateless design degrades gracefully and the stateful one fails hard. Note that polling uses MORE requests, not fewer - that is the cost being accepted.

Q2. The rule "one vote per device per poll" is enforced BOTH by an if-check in the request handler AND by a UNIQUE constraint in the database. Why is the if-check alone insufficient?

Correct answer: C) Two simultaneous requests can both pass the check before either writes, so only the constraint actually prevents a double vote

Why: There is a window between reading and writing that no application code can close. Two taps in the same millisecond both run the SELECT, both find nothing, and both INSERT. The database serialises the write, so the second raises IntegrityError, which the handler catches and turns into the same friendly 409. The if-check exists for the message; the constraint is what makes the rule true.

Q3. What is the main purpose of an AGENTS.md file in a repository?

Correct answer: D) To state the stack, test commands and hard rules in a durable place every agent re-reads

Why: Agents follow explicit written instructions reliably and infer unwritten preferences poorly. An absence is not a signal: a codebase that happens to avoid WebSockets will not teach an agent to avoid them. Unlike a chat message, a file is version controlled, shared across tools and teammates, and re-read on every task rather than falling out of a context window.

Q4. When reviewing a feature written by a coding agent, the test suite is fully green. Why is that NOT sufficient evidence that the requirements are met?

Correct answer: B) The agent wrote the tests too, so a rule it forgot has no test and nothing reports the absence

Why: A suite written by the same process that wrote the code tests what the code does, not what the spec required. Coverage of requirements is the only coverage that matters here, and only a numbered spec can reveal a gap. This is why every acceptance criterion should be numbered and checked one by one against the diff.

Q5. In the GitOps model used in this session, what actually triggers a deployment to the cluster?

Correct answer: D) CI committing the new image tag to the repository, which ArgoCD observes and reconciles

Why: The repository is the desired state and ArgoCD continuously makes the cluster match it. CI holds no cluster credentials at all - its most privileged action is writing a commit. Pulling rather than pushing keeps the credential inside the cluster, and makes git log a complete, revertible deployment history.

Q6. A path-filtered CI pipeline reports a green check on a commit that changed only documentation. What does that green check actually mean?

Correct answer: C) No relevant paths changed, so the test jobs were skipped and nothing ran

Why: Path filtering is correct and desirable in a monorepo, but it changes what a green tick means. "Nothing relevant changed" and "the tests pass" look identical in the UI. Before trusting a green check - especially during a live demo - confirm that the jobs you care about actually executed.

Q7. What most distinguishes a usable specification from a prompt when handing work to a coding agent?

Correct answer: D) It has numbered, individually checkable acceptance criteria, plus where code lands, what to reuse and what is out of scope

Why: Numbering creates a shared address space: "rule 6 has no test" is unambiguous and verifiable, while "the rate limiting seems incomplete" merely starts a discussion. Naming what to reuse prevents a second polling hook or fetch wrapper appearing, and an explicit non-goals list stops an agent expanding scope out of helpfulness.

Q8. Why should a Kubernetes manifest reference a container image by commit SHA rather than the :latest tag?

Correct answer: C) A SHA tag is immutable, so you can say exactly what is running and roll back with confidence

Why: The :latest tag moves, so the version you would roll back TO has moved as well, and a runtime may serve a cached image while appearing to deploy a new one - a failure mode hit during this very build. Immutable tags turn deployment history into a fact rather than a rumour; digests are stronger still.

Q9. The Pulse web app only ever calls relative paths such as /api/polls, with no configurable API base URL. What does this design eliminate?

Correct answer: B) A per-environment build, and the class of bugs where the wrong backend URL is shipped

Why: The usual approach bakes an API URL into the bundle at build time, which means rebuilding per environment and, eventually, shipping the staging bundle to production. With relative paths there is no value to set incorrectly. The trade is that a router must sit in front in every environment - the dev server proxy locally, an ingress or nginx in the cloud - which you already run anyway.

Q10. Pulse identifies voters by a random device_id in localStorage, which any user can trivially reset. How should this design decision be characterised?

Correct answer: C) A deliberate trust boundary left undefended because the defence costs more than the threat

Why: The threat is one attendee nudging a bar chart in a room where the result changes nothing. Every alternative costs more than that: email login kills participation outright, IP address gives an entire venue behind one NAT a single vote, and fingerprinting is privacy-hostile while still being defeatable. Being deliberate about an exposure is different from being unaware of it, and naming it precisely is the engineering skill being tested.